nopCommerce + OpenTelemetry

Checkout Flow Observability

nopCommerce Architecture

Nop.Web

HTTP entry point — Controllers, Views

Nop.Web.Framework

DI, base infrastructure, middleware

Nop.Services

Business logic — OrderProcessingService

Nop.Data

Repositories, EF Core, migrations

Nop.Core

Entities, interfaces, events (base)

Why instrument here?

→ ASP.NET auto-instrumentation already covers HTTP spans in Nop.Web

→ SqlClient auto-instrumentation covers SQL queries in Nop.Data

→ Nop.Services is the gap — payment, persistence, events are orchestrated here

→ PlaceOrderAsync is the single method that coordinates the entire checkout

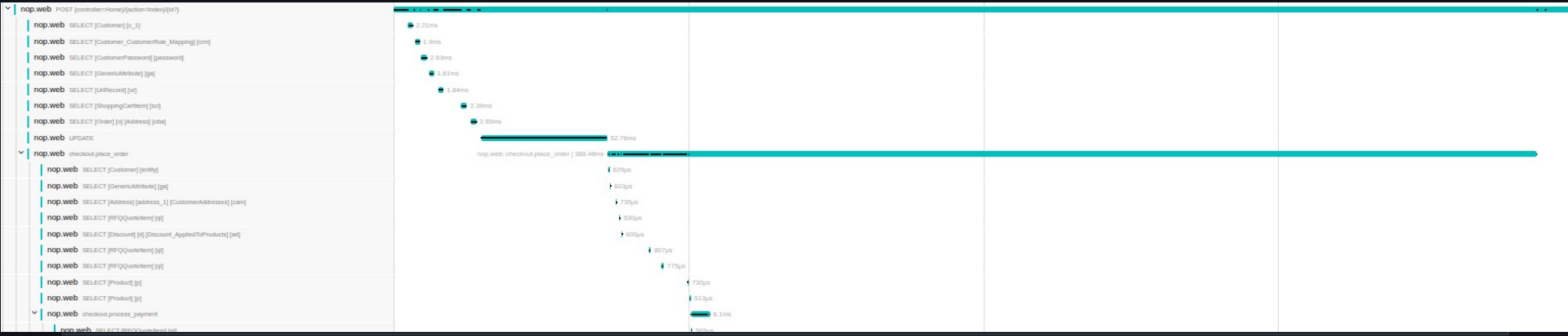
→ Surgical: 3 files changed, business logic untouched

Instrumentation Decision

- Chose "Customer places an order" — the most business-critical flow; if checkout fails, the store loses money
- Instrumented at OrderProcessingService.PlaceOrderAsync — where payment, persistence, notifications and events are orchestrated in a single method
- ASP.NET auto-instrumentation covers HTTP. SqlClient covers SQL. I only needed to instrument what's in between
- Surgical approach: 3 files changed (OrderProcessingService.cs, Program.cs, NopTelemetryConstants.cs), business logic untouched, zero overhead if OTel SDK is absent

End-to-End Tracing

POST /checkout/OpcConfirmOrder	auto - ASP.NET
checkout.place_order	custom
checkout.process_payment	custom
checkout.save_order	custom
SQL queries	auto - SqlClient
checkout.send_notifications	custom
checkout.publish_event	custom



Included: order.total, payment.method, order.items | Excluded: emails, addresses, names, card numbers (zero PII)

3 Custom Metrics — The 2am Test

Metric	Type	If it fires at 2am, the on-call engineer knows…
`nop.checkout.place_order.attempts`	Counter	Checkout volume dropped or spiked abnormally — checkout may be unreachable
`nop.checkout.place_order.failures`	Counter	The checkout pipeline is broken — users cannot complete orders
`nop.checkout.place_order.duration`	Histogram	DB or payment gateway is degraded — p95 rising before visible errors appear

attempts + failures → error rate in Grafana | duration → detects degradation before failures become visible

Privacy — Defence in Depth

In code

- Span attributes are strictly operational: order.total, payment.method, order.items
- No emails, addresses, names or card numbers are ever recorded
- Explicit decision for every attribute — full control at the point of instrumentation

OTel Collector — pii-redact processor

- Deletes before export: authorization headers, cookies, emails, SQL statements, query strings, request bodies
- Why here? Auto-instrumentation (ASP.NET, SqlClient) can capture PII without my control — the Collector is the last barrier
- Even if something escapes in code, it is filtered here before reaching Jaeger or Prometheus

Rejected alternative: filtering only in code — fragile, auto-instrumentation escapes our control

What I Did NOT Instrument — and Why

Not instrumented	Reasoning
Controllers	ASP.NET auto-instrumentation already covers all HTTP spans
Generic EventPublisher	Would affect the entire system (login, newsletter...) — instrumented only the OrderPlacedEvent
Plugins (Brevo, Avalara)	Independent modules, out of checkout scope
Cache (IStaticCacheManager)	Overhead on thousands of operations per request — noise with no value

Internal checkout methods are protected virtual — subclass override would be possible but fragile and disproportionate. Chose direct surgical change in the service.

Live Demo

Load Test

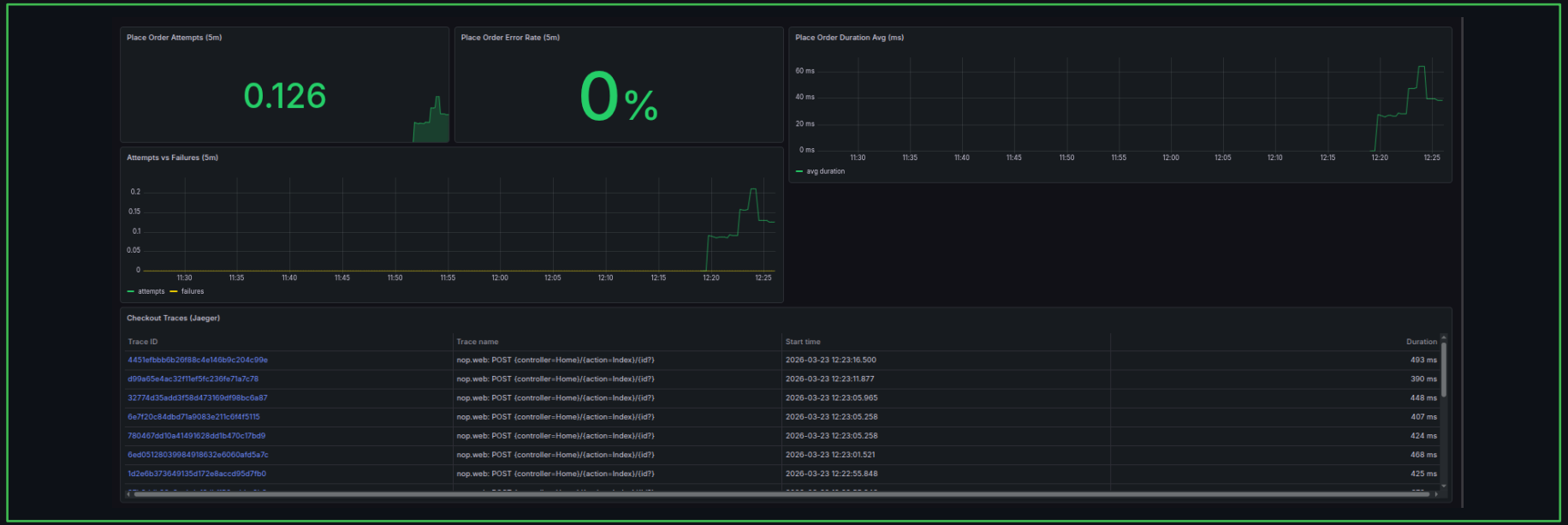
```
k6 run loadtest/checkout-flow.js
```

Grafana

```
localhost:3000
```

Jaeger

```
localhost:16686
```



Backup screenshots — real demo runs live with k6 generating traffic

Live Demo

Load Test

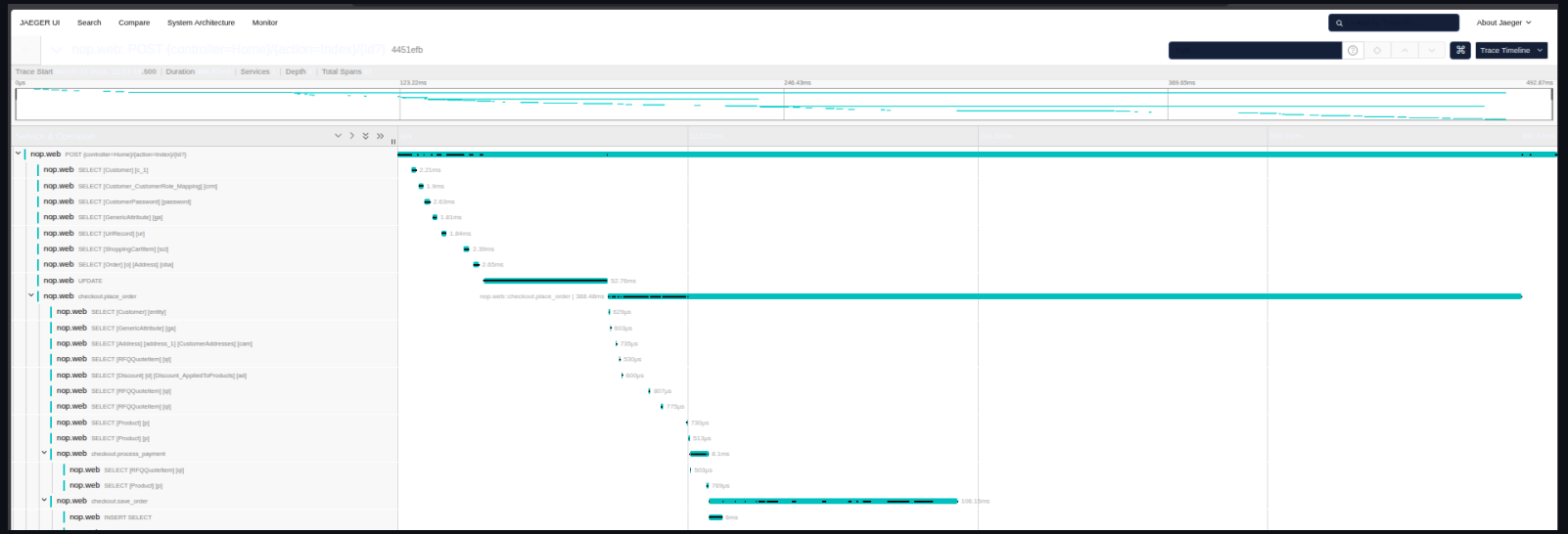
```
k6 run loadtest/checkout-flow.js
```

Grafana

`localhost:3000`

Jaeger

`localhost:16686`



Backup screenshots — real demo runs live with k6 generating traffic

Live Demo

Load Test

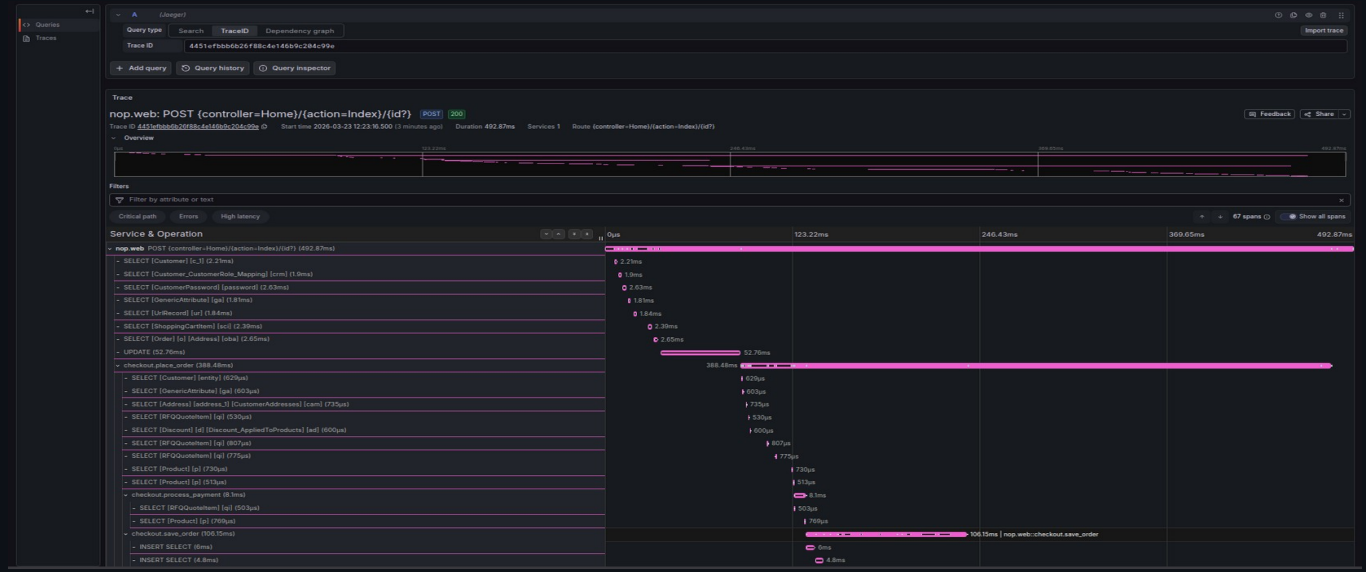
```
k6 run loadtest/checkout-flow.js
```

Grafana

```
localhost:3000
```

Jaeger

```
localhost:16686
```



Backup screenshots — real demo runs live with k6 generating traffic